

Dokumentacja Projektowa: Skarbonki

1 Opis problemu i opis rozwiązania

1.1 Opis problemu

Zadanie "**Skarbonki**" polega na określeniu minimalnej liczby skarbonek, które należy rozbić młotkiem, aby uzyskać dostęp do zawartości wszystkich n skarbonek, mając do dyspozycji informacje o rozmieszczeniu kluczy. W każdej skarbonce może znajdować się wiele kluczy, ale do każdej pasuje dokładnie jeden.

Modelując ten problem za pomocą teorii grafów, każda skarbonka jest reprezentowana jako wierzchołek, a relacja "klucz w skarbonce A otwiera skarbonkę B" jako krawędź. Ponieważ każda skarbonka ma dokładnie jeden klucz otwierający, graf dzieli się na słabo spójne składowe. W każdej takiej składowej znajduje się dokładnie jeden cykl. Rozbicie dowolnej skarbonki z cyklu zapoczątkowuje reakcję łańcuchową otwierającą wszystkie skarbonki w danej składowej. Rozwiązanie problemu sprowadza się zatem do policzenia liczby słabo spójnych składowych w grafie.

1.2 Opis rozwiązania (Pseudokod)

Aby zliczyć słabo spójne składowe, graf skierowany traktowany jest w implementacji jako graf nieskierowany. Wykorzystany jest algorytm przeszukiwania w głąb (DFS).

```
Funkcja DFS_Skarbonki():
    licznik_spojnych_skladowych = 0
    Dla kazdego wierzchołka i w grafie:
        visited[i] = false

    Dla kazdego wierzchołka x w grafie:
        Jezeli x nie jest odwiedzony (visited[x] == false):
            Zwiększ licznik_spojnych_skladowych o 1
            Uruchom DFS_1(x)

    Zwroc licznik_spojnych_skladowych

Funkcja DFS_1(wierzchołek v):
    Oznacz v jako odwiedzony (visited[v] = true)
    Dla kazdej krawedzi e wychodzącej z v:
        Niech u będzie wierzchołkiem na drugim końcu krawedzi e
        Jezeli u nie jest odwiedzony (visited[u] == false):
            Uruchom DFS_1(u)
```

2 Opis użytych struktur danych

Rozwiązanie bazuje na strukturach danych zaimplementowanych podczas zajęć:

- **Vertex:** Klasa reprezentująca wierzchołek. Przechowuje unikalny numer wierzchołka.
- **Edge:** Klasa reprezentująca krawędź łączącą dwa wierzchołki. Udostępnia metodę `Mate(v)` zwracającą wierzchołek na drugim końcu krawędzi.
- **GraphAsMatrix:** Klasa reprezentująca graf. Wewnętrznie przechowuje:
 - `std::vector<Vertex*> vertices`- wektor wskaźników na wszystkie wierzchołki.

- `std::vector<std::vector<Edge*> > adjacencyMatrix`- macierz sąsiedztwa przechowująca wskaźniki na obiekty krawędzi.
- `std::vector<bool> visited`- wektor wartości logicznych służący do monitorowania odwiedzonych wierzchołków podczas przeszukiwania.
- **Iteratory** (`Iterator<T>`): Zaimplementowano klasy iteratorów ułatwiające przeglądanie struktury grafu:
 - `AllVerticesIter`- służy do przejścia przez wszystkie wierzchołki w grafie w funkcji głównej algorytmu.
 - `EmanatingEdgesIter`- służy do sekwencyjnego przeglądania wszystkich krawędzi wychodzących z zadanego wierzchołka w algorytmie rekurencyjnym.

3 Oszacowanie złożoności czasowej i pamięciowej użytych struktur danych

3.1 Złożoność pamięciowa

- **Macierz sąsiedztwa** (`adjacencyMatrix`): Przechowuje wskaźniki dla każdej potencjalnej pary wierzchołków, co daje złożoność pamięciową rzędu $O(V^2)$, gdzie V to liczba wierzchołków.
- **Wektory pomocnicze** (`vertices`, `visited`): Ich rozmiar jest wprost proporcjonalny do liczby wierzchołków, złożoność to $O(V)$.
- **Krawędzie**: Tworzonych jest E obiektów krawędzi (gdzie $E = V$), co wymaga $O(E)$ pamięci.

Całkowita złożoność pamięciowa zdominowana jest przez macierz sąsiedztwa i wynosi $O(V^2)$.

3.2 Złożoność czasowa podstawowych operacji

- **Dodanie wierzchołka**: Złożoność $O(1)$ dzięki `std::vector::push_back`.
- **Dodanie krawędzi** (`AddEdge`): Odbywa się poprzez przypisanie wskaźników w macierzy sąsiedztwa w czasie $O(1)$.
- **Pobranie iteratora krawędzi incydujących** (`EmanEdgesIter`): Utworzenie iteratora i znalezienie pierwszej krawędzi wymaga przeglądu jednego wiersza macierzy, w najgorszym wypadku czas $O(V)$. Przesunięcie na kolejną krawędź (`next()`) działa również pesymistycznie w czasie $O(V)$.

4 Oszacowanie złożoności czasowej i pamięciowej głównych algorytmów

4.1 Złożoność czasowa

Głównym algorytmem jest metoda `DFS_Skarbonki`, wykorzystująca rekurencyjną procedurę `DFS_1`. Algorytm przetwarza każdy z V wierzchołków dokładnie raz, co wynika z użycia tablicy `visited`. Z powodu wykorzystania macierzy sąsiedztwa, znalezienie wszystkich sąsiadów dla pojedynczego wierzchołka wymaga przejrzania całego wiersza długości V w macierzy. Operacja ta jest wykonywana dla każdego wierzchołka, zatem łączny czas przeglądania krawędzi wynosi $O(V \cdot V) = O(V^2)$. Proces inicjalizacji tablicy odwiedzin oraz budowa samego grafu to również odpowiednio $O(V)$ i $O(V)$. Złożoność czasowa całego algorytmu przeszukiwania wraz z konstrukcją grafu wynosi $O(V^2)$.

4.2 Złożoność pamięciowa

Złożoność pamięciowa samego algorytmu przeszukiwania zależy od stosu wywołań rekurencyjnych. W najgorszym przypadku, gdy wszystkie skarbonki są połączone w pojedynczą linię, głębokość rekurencji wyniesie V . Oznacza to dodatkową pamięć $O(V)$ na stosie. Jest to jednak przykryte przez zajętość struktur klasy, co daje łączną pamięć $O(V^2)$ dla całego procesu.

5 Dokumentacja użytkowa

5.1 Jak uruchomić program

1. Skompiluj kod źródłowy przy pomocy dowolnego kompilatora języka C++. W terminalu: `g++ main.cpp -o skarbonki`.
2. Uruchom skompilowany plik `skarbonki.exe`.

5.2 Jak wprowadzać dane

Program odczytuje dane ze standardowego wejścia. Należy podać sekwencję liczb całkowitych oddzielonych białymi znakami:

1. Pierwsza wprowadzona liczba oznacza liczbę skarbonek n ($1 \leq n \leq 1000000$).
2. Następnie wprowadza się n liczb. i -ta z kolei wprowadzona w tym bloku liczba informuje, w której skarbonce schowany jest klucz otwierający skarbonkę nr i .

Po zatwierdzeniu wprowadzonych danych, na standardowe wyjście wypisana zostanie pojedyncza liczba całkowita będąca wynikiem algorytmu.